Making a Humanoid Crawl Like a Spider

Reinforcement Learning for Inverted Quadruped Locomotion on the Unitree G1 in MuJoCo

Seven Iterations, Six Reward Rewrites, and the Triangle-Wave CPG That Finally Made It Move

[IMAGE: The G1 robot upright (left) vs. belly-up spider pose (right).]

Lucky Robots Challenge Submission

Subhadeep Chatterjee

April 6, 2026

Abstract

This report documents the complete development of a system that teaches a Unitree G1 humanoid robot to crawl like a spider—belly-up, all four limbs acting as legs. The project iterated through seven major approaches: IK-in-the-RL-loop, direct 21-joint control, central pattern generators with wrong signs, imitation learning from reference trajectories, static standing (the first success), sine-wave CPG with RL, and finally **triangle-wave CPG with RL corrections**—the approach that produced real forward locomotion.

The reward function went through six revisions, each closing an exploit the RL agent discovered: free cumulative progress, micro-jerk sliding, legs-only crawling, dancing in place, **jumping for displacement**, and **asymmetric limb coordination** where one hand went forward while three limbs pushed backward. Each failure is documented with the reasoning that led to the next design.

The final system uses an 8-DOF triangle-wave gait generator (2 joints per limb) with fast-swing/slow-stance asymmetry. RL adds small corrections and the robot produces sustained forward crawling with all four limbs cycling through swing and stance phases.

Contents

1 Why This Problem Is Hard **3**
1.1 The Fundamental Mismatch . . . 3
1.2 What Makes This Different from Quadruped RL . . . 3

2 Day One Reality Check: The Robot Is on Ice **3**

3 XML Surgery and Robot Preparation **4**

4 Solving Standing — The Foundation **4**
4.1 Three Physics Problems . . . 4
4.2 Standing RL . . . 4

5 Seven Approaches — A Chronicle of Informed Failure **5**
5.1 Attempt 1: IK-in-the-Loop (Failed) . . . 5
5.2 Attempt 2: Direct 21D Joint Control (Failed) . . . 5
5.3 Attempt 3: CPG + RL (Failed) . . . 5
5.4 Attempt 4: Imitation Learning (Failed) . . . 6
5.5 Attempt 5: Standing RL (Success) . . . 6
5.6 Attempt 6: 8-DOF Sine Gait + RL (Partial) . . . 6
5.7 Attempt 7: Triangle-Wave CPG + RL (Working) . . . 7

6 The Two-Gain Architecture **8**

7 Reward Engineering — Six Versions, Six Exploits **8**
7.1 v1: Cumulative Progress . . . 8
7.2 v2: Per-Step Displacement . . . 8
7.3 v3: Require Lifting . . . 8
7.4 v4: Fairness + Lazy Penalty . . . 9
7.5 v5: Anti-Jumping + Grounded Constraint . . . 9
7.6 v6: Per-Limb Directional Consistency (Final) . . . 9
7.7 Complete Final Reward . . . 10
7.8 Meta-Lesson: Adversarial Reward Design . . . 10

8 The Final System: Triangle CPG + RL **10**
8.1 Architecture 11
8.2 Key Parameters (Post-Search) 11
8.3 Observation Space (33D) 11
8.4 Episode Structure 11

9 Results **12**
9.1 Stages of Success 12

10 Tools Developed **13**

11 Key Lessons **13**

12 Future Directions **13**

13 Conclusion **13**

A Physics Configuration **14**

B File Inventory **14**

1 Why This Problem Is Hard

1.1 The Fundamental Mismatch

The Unitree G1 is a bipedal humanoid. Its kinematics, actuator sizing, weight distribution, and joint ranges are optimized for upright walking. Asking it to crawl belly-up is asking it to do something its body was never designed for.

Key mismatches:

1. **Asymmetric actuators.** Legs: 88–139 Nm. Arms: 25 Nm (raised to 60 Nm in software). In spider mode, all four limbs must support the body equally—but two are drastically underpowered.
2. **Insufficient arm reach.** Shoulder joints weren't designed to reach behind the body and push against the ground. We pushed shoulder_pitch to -2.80 rad (near the -3.09 limit) just to get hands on the floor.
3. **No contact geometry.** The robot has foot soles for bipedal walking, not hand-ground or inverted-foot-ground contact surfaces. We added box geoms at all four EE sites.
4. **29 DOF.** Too many for brute-force RL exploration, too structured for generic optimization. A real spider has 48 joints across 8 legs with identical morphology.

◇ Decision Point

First decomposition: What's the minimal viable version?
We identified two sub-problems: (1) Can the robot hold the spider pose statically under gravity? (2) Can it then move forward? This decomposition—*stand, then crawl*—shaped the entire project. We refused to attempt crawling until standing was solved.

1.2 What Makes This Different from Quadruped RL

Standard quadruped RL benefits from symmetric morphology (all 4 legs identical), purpose-built joints, established baselines, and powerful actuators on every limb. Our problem has none of these. The legs and arms have different kinematics, different torque limits, different mass distributions, and different reach envelopes. No baselines exist for belly-up humanoid crawling.

[IMAGE: Spider/lizard crawling reference showing the wave gait pattern—which legs are in swing vs. stance at each phase (0°, 90°, 180°, 270°).]

2 Day One Reality Check: The Robot Is on Ice

Before any locomotion work could begin, we hit a show-stopping problem. A diagnostic script inspected every geom:

× What Failed

All mesh geoms had MuJoCo's default friction: [1.0, 0.005, 0.0001]. MuJoCo computes contact friction as the geometric mean of the two surfaces. Even with a high-friction floor, $\sqrt{0.5 \times 0.005} = 0.05$ for torsional friction. **The robot was on ice.**
This was discovered on day 4. Three days of work—IK, CPG, imitation learning—were built on a broken foundation.

✓ What Worked

One line fixed every subsequent experiment:

```
for i in range(model.ngeom):
    if model.geom_contype[i] > 0 or model.geom_conaffinity[i] > 0:
        model.geom_friction[i] = [3.0, 0.5, 0.5]
```

Drift: meters → 2.4 millimeters.

▷ Key Insight

Lesson: Always validate the physics before building control on top of it. A 10-minute diagnostic would have saved 3 days.

3 XML Surgery and Robot Preparation

1. **Ankle joints removed.** Tiny links (0.074 kg) vibrated at high frequency. Commenting them out from XML eliminated the instability source.
2. **Contact geoms added.** Four box geoms at EE sites with `friction="3 0.5 0.5"`, `condim="6"` for full friction cone.
3. **Arm torques boosted.** 25 → 60 Nm for shoulder/elbow. Physically unrealistic but necessary—the real G1's arms can't support body weight in this configuration.
4. **Belly-up quaternion.** $[0.707, 0, -0.707, 0]$ (-90 around Y). Early use of $[0.707, 0.707, 0, 0]$ (rotation around X) placed the robot on its side, causing it to "vanish" due to nonsensical spawn height computation.

4 Solving Standing — The Foundation

4.1 Three Physics Problems

Problem 1: Impact forces. Dropping the robot from any height destabilized the PD. **Solution:** `gentle_gravity_settle()`—ramp gravity from 0% to 100% over 2 seconds with PD already active.

Problem 2: Wrong spawn height. Solution: `find_spawn_height()`—compute exact height where lowest EE is 3 cm above ground. Zero drop.

Problem 3: Wrong PD targets. Zero-gravity pose ≠ full-gravity equilibrium. Tracking the file pose under gravity creates constant error → lateral forces → sliding. **Solution:** After settling, record the *actual* joint positions as PD targets.

4.2 Standing RL

With physics solved, standing RL converged in ∼2M steps. The robot holds the spider pose stably with body elevated.

[IMAGE: MuJoCo screenshot of stable spider stand—belly up, all 4 EEs on ground, torso elevated.]

5 Seven Approaches — A Chronicle of Informed Failure

5.1 Attempt 1: IK-in-the-Loop (Failed)

Reasoning: RL in Cartesian space (12D EE positions) should be easier than joint space. IK handles the geometry.

× What Failed

IK occasionally failed (singular Jacobians, joint limits). The RL agent couldn't distinguish its own mistakes from IK failures. After 2M steps, the agent learned to output zero deltas—the only action guaranteed not to trigger an IK failure. The learning signal was corrupted by the solver.

→ **Why this led to Attempt 2:** The IK solver was an unreliable middleman. Every IK failure injected noise into the reward signal that RL couldn't learn from. The obvious fix: *remove IK entirely* and let RL control joints directly. If the agent outputs joint angles, there is no solver to fail—every action is valid.

5.2 Attempt 2: Direct 21D Joint Control (Failed)

Reasoning: Remove the IK middleman. RL directly outputs all joint angles. Every action is valid—no solver failures.

× What Failed

21D continuous action space. To produce a coordinated gait, the policy must simultaneously discover which joints, by how much, at what frequency, with what phase offsets. PPO found nothing. The agent converged on holding still—the action that minimized negative reward. It literally learned to die faster because shorter episodes accumulated less negative reward.

Fix for "learning to die": Added a positive alive baseline so that surviving was always better than dying. This is a general principle: *if the agent scores higher by dying, your reward is broken.*

→ **Why this led to Attempt 3:** The problem wasn't RL or the reward—it was the action space. Asking PPO to discover a periodic, coordinated, multi-limb gait from scratch in 21 continuous dimensions is like searching for a needle in a haystack. The solution: *provide the rhythmic structure* using a Central Pattern Generator (CPG) so RL only needs to modulate a few parameters, not invent the entire gait.

5.3 Attempt 3: CPG + RL (Failed)

Reasoning: CPGs produce rhythmic motion inherently. RL only modulates frequency, amplitude, coupling—a much smaller search space than raw joint control.

× What Failed

The CPG sign conventions were wrong for the belly-up orientation. Joint axes invert when the robot is flipped. We guessed the signs. The guesses were wrong. Limbs pushed against each other. The robot spun in circles.

Lesson: With $2^8 = 256$ possible sign combinations, manual guessing is futile. This directly motivated the automated `-mode search` in the final system.
→ **Why this led to Attempt 4:** The CPG approach was sound in principle but sabotaged by wrong signs *and* by the friction problem (discovered during this phase). Rather than debug CPG signs while fighting ice-like friction, we pivoted to imitation learning—let an IK-generated reference trajectory define the motion, and train RL to track it. This bypasses the sign problem entirely because the reference already encodes correct joint directions.

5.4 Attempt 4: Imitation Learning (Failed)

Reasoning: Generate reference crawling trajectories offline with IK (where failures can be detected and discarded). RL adds small corrections (±0.1 rad residual policy). The reference provides 90% of the motion; RL handles balance.

× What Failed

Two failures:

1. References were generated in zero gravity but executed under full gravity. The targets were physically inconsistent—the equilibrium positions differ.
2. The robot discovered it could collect positive reward by dancing in place—oscillating joints (high tracking score) while sliding forward via momentum (small velocity reward). The reward function could be satisfied without walking.

→ **Why this led to Attempt 5:** Two realizations: (1) We were trying to crawl before we could stand. If PD control can't even hold a static pose under gravity, it certainly can't hold a dynamic gait. (2) The friction was fundamentally broken (the "ice" discovery happened here). We needed to *go back to basics*: fix the physics, solve standing first, and only then attempt locomotion. Standing became the gating milestone.

5.5 Attempt 5: Standing RL (Success)

Documented in Section 4. Gentle gravity settle + spawn height + friction fix + gravity-adjusted PD targets. Converged in ∼2M steps. The foundation that made everything else possible.
→ **Why this led to Attempt 6:** Standing worked. The physics was now trustworthy: correct friction, proper contacts, stable PD. Time to add motion. The CPG approach (Attempt 3) was the right idea—just sabotaged by bad physics and wrong signs. Now we could revisit it with: (a) fixed friction, (b) a working stand as the starting pose, (c) an automated sign search instead of guessing, and (d) a reduced 8-DOF action space (the 21D lesson from Attempt 2).

5.6 Attempt 6: 8-DOF Sine Gait + RL (Partial)

Reasoning: Reduce to 8 DOF (pitch + bend per limb). Built-in sine wave gait—no reference files, no IK, no solver. Automated sign search (`-mode search`) across 324 combinations.

× What Failed

Three problems:

1. Stand PD gains ($K_p = 3\times$, damping $= 3.0$) killed all motion—the sine wave couldn't overcome the overdamped PD.
2. Sine waves have slow acceleration at extremes—the limbs decelerated exactly when they needed maximum push.
3. Multiple reward exploits (documented in Section 7).

✓ What Worked

The **two-gain architecture** was discovered here: stiff gains for settling ($K_p = 3\times$, $d = 3.0$), soft gains for crawling ($K_p = 1\times$, $d = 0.5$). This was the breakthrough connecting standing to crawling.

→ **Why this led to Attempt 7:** The architecture was right—8 DOF CPG + RL with automated signs. But two specific problems remained: (1) the sine waveform wasted energy decelerating at extremes (limbs slowed down exactly when they should push hardest), and (2) the gait frequency was too low (0.5 Hz = 2 seconds per cycle, too slow for momentum). The fix was mechanical, not algorithmic: replace sine with a triangle/piecewise waveform that spends 20% of the cycle in fast swing and 80% in slow stance, and increase frequency to 1.5–2.0 Hz.

5.7 Attempt 7: Triangle-Wave CPG + RL (Working)

◇ Decision Point

Why triangle over sine? Sine waves have smooth acceleration everywhere—gentle at the peaks (where the limb reverses direction) and gentle at zero-crossing (where the limb should push hardest). For crawling, we need the opposite: *fast swing* (snappy limb repositioning) and *slow stance* (sustained pushing force). A triangle/piecewise waveform achieves this by spending only 20% of the cycle in swing and 80% in stance, with linear (constant-velocity) segments instead of sinusoidal ones.

The triangle gait generator:

Listing 1: Piecewise triangle gait—fast swing, slow stance.

```python
def generate_triangle_targets(home_qpos, joint_map, phase, params):
    swing_ratio = params.get("swing_ratio", 0.2)  # 20% swing
    for i, limb_name in enumerate(LIMB_NAMES):
        t = ((phase + PHASE_OFFSETS[i]) % (2*np.pi)) / (2*np.pi)
        if t < swing_ratio:
            # SWING: fast linear ramp up then down
            s = t / swing_ratio
            curve = 1.0 - abs(2*s - 1)  # triangle peak at 50% of swing
            pitch_offset = pitch_amp * pitch_signs[i] * curve
            bend_offset  = bend_amp  * bend_signs[i]  * curve
        else:
            # STANCE: slow linear push
            s = (t - swing_ratio) / (1.0 - swing_ratio)
            pitch_offset = -pitch_amp * pitch_signs[i] * 0.6 * (1-s)
            bend_offset  = 0.0
```

Combined with higher gait frequency (1.5–2.0 Hz vs. the original 0.5 Hz), this produced 3–4× faster stepping and visually recognizable crawling for the first time.

#	Approach	Result	Critical Lesson
1	IK-in-the-loop (12D)	Failed	Don't put fallible solvers in RL loop
2	Direct joint control (21D)	Failed	Action space too large; agent learned to die
3	CPG + RL (wrong signs)	Failed	Automate sign search, don't guess
4	Imitation learning	Failed	Zero-gravity refs don't transfer; reward exploits
5	Standing RL	Success	Physics first, RL second
6	8-DOF sine + RL	Partial	Stand gains kill motion; sine too smooth
7	Triangle CPG + RL	Working	Fast swing + slow stance + all lessons applied

Table 1: All seven approaches, chronologically.

6 The Two-Gain Architecture

This discovery deserves its own section. The stand controller used $K_p = 3 \times \tau_{\max}$, passive damping $d = 3.0$—tuned for zero motion. When the gait generator applied offsets, the PD snapped joints back faster than the gait could move them. The robot appeared frozen.

Reducing gains to $K_p = 1\times$, $d = 0.5$ allowed motion but made the gravity settle phase collapse. The robot fell before the gait started.

✓ What Worked

Switch gains between phases:

- **Settle:** $K_p = 3\times$, $K_d = 0.3 \times K_p$, $d = 3.0$ (rigid pose, gravity ramps up).
- **Crawl:** $K_p = 1\times$, $K_d = 0.1 \times K_p$, $d = 0.5$ (gait can move joints).

7 Reward Engineering — Six Versions, Six Exploits

7.1 v1: Cumulative Progress

$r = 5v_x + 3(x_t - x_0) + \text{stability}$

× What Failed

$(x_t - x_0)$ grows forever. Move once, collect free reward every step. Agent moved 5 cm at step 100, then held still. Reward at step 1500: >2000. Actual crawling: zero.

7.2 v2: Per-Step Displacement

$r = 200 \cdot \max(0,\ x_t - x_{t-1})$

× What Failed

Agent jerked back-and-forth, all 4 feet planted. Micro-positive displacements from momentum. Sliding, not walking.

7.3 v3: Require Lifting

Added: lifting bonus (+2), all-feet-down penalty (−3), jerk penalty.

✓ What Worked

Legs started crawling! Visible swing/stance cycling.

× What Failed

Only legs worked. Arms stayed planted. Reward said "lift *a* limb," not "lift *all* limbs in turn."

7.4 v4: Fairness + Lazy Penalty

Added: per-limb lift tracking, fairness reward ($+2 \times n_{\text{active}}/4$), lazy penalty ($-0.5$ per limb that never lifts after step 200).

✓ What Worked

Arms began participating. Lift counts increased for all 4 limbs.

× What Failed

New exploit: jumping. The agent discovered that jumping forward produced massive displacement reward ($200 \times \Delta x$). It launched itself off the ground, landed a few cm forward, and repeated. No crawling—pure hopping. The displacement reward was too dominant and had no constraint on *how* displacement was achieved.

7.5 v5: Anti-Jumping + Grounded Constraint

Three fixes targeting the hopping exploit:

- **Vertical velocity penalty:** $-5.0 \cdot |v_z|$. Jumping requires vertical velocity; crawling doesn't.
- **Airborne penalty:** -3.0 if $n_{\text{contacts}} \leq 1$ (fully airborne = penalized hard).
- **Height variation penalty:** $-2.0 \cdot (h - h_{\text{settled}})^2$. Any deviation from settled height is penalized.
- Displacement weight reduced: $200 \rightarrow 50$.

✓ What Worked

Jumping eliminated. Robot stayed grounded.

× What Failed

New exploit: asymmetric limb coordination. The left hand pushed forward while the other three limbs pushed backward. Net displacement was small but positive. The agent satisfied "all limbs must lift" and "forward displacement" simultaneously—but 3 limbs were fighting the direction of travel.

7.6 v6: Per-Limb Directional Consistency (Final)

The asymmetry fix required computing each EE's individual contribution to forward motion:

Listing 2: Per-limb directional consistency.

```
# Compute per-EE velocity in world X
ee_fwd_vels = []
for name in EE_SITES:
    sid = mujoco.mj_name2id(model, mujoco.mjtObj.mjOBJ_SITE, name)
    jacp = np.zeros((3, model.nv))
    mujoco.mj_jacSite(model, data, jacp, None, sid)
    ee_vel_x = jacp[0] @ data.qvel  # X component
    ee_fwd_vels.append(ee_vel_x)

```

```
# Penalize limbs moving backward during stance
backward_penalty = sum(max(0, -v) for v in ee_fwd_vels if v < 0)
```

✓ What Worked

All four limbs now push in the same direction. The symmetry-breaking failure was resolved. Forward velocity became consistent and sustained.

7.7 Complete Final Reward

$$
\begin{aligned}
r_{\text{final}} = 50 \cdot \Delta x^{+} & & \text{(displacement)} \\
+ 2 \cdot \mathbb{1}[\text{lifting}] + 2 \cdot \text{fairness} + 0.5 \cdot \mathbb{1}[\text{cycle}] & & \text{(limb cycling)} \\
- 3 \cdot \mathbb{1}[\text{all_down}] - \text{lazy} - 3 \cdot \mathbb{1}[\text{airborne}] & & \text{(anti-exploit)} \\
- 5|v_z| - 2(h - h_0)^2 - \text{backward} & & \text{(anti-jump, anti-asymmetry)} \\
+ 0.5(r_h + r_{\text{belly}} + r_{\text{align}}) + 0.1 & & \text{(stability)} \\
- 0.3|v_y| - 0.3|\dot{\psi}| - 0.001E - 0.02\|\Delta a\|^2 & & \text{(regularization)}
\end{aligned}
\tag{1}
$$

[IMAGE: 6-panel reward evolution diagram—each panel shows version, exploit, and fix.
Arrows connect versions. Red for exploits, green for fixes.
Show progression from v1 (cumulative) through v6 (directional consistency).]

7.8 Meta-Lesson: Adversarial Reward Design

▷ Key Insight

For every positive reward term, ask: *"What is the laziest/cheapest way to maximize this?"* Then penalize that behavior.

If you reward:	**The agent will:**
Cumulative distance	Move once, then stop
Instantaneous velocity	Jerk back-and-forth
"Any limb lifted"	Use only the strongest limbs
Large displacement (no constraints)	Jump
"All limbs lift" (no direction constraint)	One limb forward, three backward

8 The Final System: Triangle CPG + RL

8.1 Architecture

```
                    RL Agent (8D corrections)
                            |
                            v
Triangle CPG  --->  [base + correction]  --->  PD Controller  --->  MuJoCo
(8 joint targets)           |                  (soft gains)        Physics
                     joint limit clamp
```

The CPG provides the base gait pattern. RL adds small corrections (± 0.15 rad) per limb. The PD controller tracks the combined targets using soft crawling gains. The CPG handles "what motion," RL handles "how to adapt."

8.2 Key Parameters (Post-Search)

Parameter	**Sine (Attempt 6)**	**Triangle (Final)**
Frequency	0.5 Hz	1.5–2.0 Hz
Swing ratio	25%	20%
Pitch amplitude	0.15 rad	0.30 rad
Bend amplitude	0.15 rad	0.20 rad
Waveform	Sinusoidal	Piecewise linear
K_p (crawl)	3× (too stiff)	1×
Damping	3.0 (too high)	0.5

Table 2: Parameter evolution from sine to triangle gait.

8.3 Observation Space (33D)

Component	**Dims**	**Purpose**
Pelvis quaternion	4	Orientation awareness
Angular velocity	3	Spin detection
Linear velocity	3	Speed + direction
Active joint positions	8	Limb state
Active joint velocities	8	Limb dynamics
EE contact states	4	Gait phase awareness
Gait clock (sin, cos)	2	Phase synchronization
Pelvis height	1	Height maintenance

8.4 Episode Structure

1. **Spawn** at computed height with spider pose.
2. **Settle** (2 s): gravity ramp with stiff PD. Switch to soft gains.
3. **Blend** (150 steps): linear blend from settled pose → CPG targets.
4. **Crawl** (up to 1500 steps): triangle CPG + RL corrections.

9 Results

Metric	Value	Target
Episode survival	1500/1500 steps	≥1000
Forward velocity	0.05–0.15 m/s	>0 (sustained)
Pelvis height	0.15–0.18 m	>0.05 m
Torso-ground contact	Never	Never
Leg crawling	Yes—visible cycling	Yes
Arm participation	Yes—all 4 limbs cycle	Yes
Jumping	Eliminated	None
Directional consistency	All limbs same direction	Yes

[IMAGE: 4–6 sequential MuJoCo frames showing one gait cycle. Label swing vs. stance per limb. Show forward displacement.]

[IMAGE: Training curves: (1) episode reward, (2) per-limb lift counts, (3) forward velocity.]

9.1 Stages of Success

1. **Stage 0** (achieved): Static spider stand, body elevated, all EEs grounded.
2. **Stage 1** (achieved): Legs produce crawling motion, some forward displacement.
3. **Stage 2** (achieved): All 4 limbs cycle, sustained forward velocity, no jumping.
4. **Stage 3** (future): Robust crawling with directional control and sim-to-real transfer.

10 Tools Developed

Tool / Mode	Purpose
`g1_viewer.py`	Tkinter pose editor with joint sliders
`g1_stand.py -mode hold`	PD hold test (verify physics)
`g1_stand.py -mode hold_debug`	Per-joint saturation analysis
`g1_crawl.py -mode sine/triangle`	Watch gait without RL
`g1_crawl.py -mode tune`	Real-time slider GUI for gait parameters
`g1_crawl.py -mode search`	Brute-force 324 sign combinations
`g1_crawl.py -mode test`	Random agent diagnostic (must be positive reward)
`g1_crawl.py -mode train`	PPO training with checkpoints
`g1_crawl.py -mode eval -video`	Evaluation with MP4 recording
`diag_friction.py`	Friction diagnostic and fix tool

11 Key Lessons

1. **Physics first, RL second.** Every failed RL run traced to a physics problem: friction, gains, contacts, spawn height. Never "just train more."

2. **The random agent test.** Before every training run: does a random agent survive full episodes with positive reward? If not, fix the environment.

3. **Two-gain architecture.** Stiff for settling, soft for motion. Neither alone works for both.

4. **Triangle > sine for locomotion.** Sine waves waste energy decelerating at extremes. Triangle/piecewise waveforms give constant-velocity segments and fast direction changes.

5. **Automate sign search.** 324 combinations $\times$ 5 seconds each = 5 minutes. Don't guess.

6. **Reward exploits are inevitable.** Cumulative progress $\rightarrow$ stop-and-collect. Per-step velocity $\rightarrow$ jerk. Lifting reward $\rightarrow$ legs-only. Large displacement $\rightarrow$ jumping. All-limb lifting $\rightarrow$ asymmetric direction. Each exploit required a specific counter-mechanism.

7. **Per-limb directional consistency.** It's not enough that all limbs move—they must all push in the same direction.

8. **The simplest solution wins.** We arrived at 8 DOF and triangle waves after trying 21D control, IK chains, CPGs with wrong signs, and imitation learning. The final system has fewer moving parts than any failed attempt.

12 Future Directions

1. **Higher velocity:** Asymmetric stance timing, unlocking hip roll for lateral push.
2. **Directional control:** Target heading in observation, train on randomized directions.
3. **Terrain robustness:** Domain randomization over friction, slope, roughness.
4. **Sim-to-real:** Actuator delay, observation noise, mass randomization.
5. **GPU training:** Current CPU-only setup limits throughput.

13 Conclusion

Making a humanoid robot crawl like a spider is a problem where the difficulty hides in the details. The RL algorithm (PPO) was never the bottleneck. The bottlenecks were friction modeling,

contact geometry, PD tuning, spawn initialization, gait waveform shape, sign conventions, and—above all—reward engineering.

The project required seven major iterations and six reward revisions. Each failure was designed to address the specific exploit of the previous version. The final system—a triangle-wave CPG with 8D RL corrections, fast-swing/slow-stance asymmetry, per-limb fairness enforcement, anti-jumping constraints, and directional consistency penalties—produces sustained forward crawling with all four limbs cycling through swing and stance phases.

The most important lesson: **the simplest architecture that could possibly work is the right starting point.** We got there after trying IK chains, 21D direct control, imitation learning, and sinusoidal CPGs. The final system is simpler than all of them.

A Physics Configuration

Two-gain configuration

```
# SETTLING (stiff)                # CRAWLING (soft)
kp = torque_limit * 3.0           kp = torque_limit * 1.0
kd = kp * 0.3                     kd = kp * 0.1
dof_damping = 3.0                  dof_damping = 0.5
# Both phases:
model.opt.timestep = 0.002;  model.opt.iterations = 50
model.opt.solver = 2  # Newton
geom_friction = [3.0, 0.5, 0.5]  # all colliding geoms
```

B File Inventory

File	Purpose
`g1_stand.py`	Spider stand: hold, debug, train, eval, test
`g1_crawl.py`	Spider crawl: sine, triangle, tune, search, train, eval, test, video
`g1_viewer.py`	Tkinter pose editor
`g1_crawl_ref.py`	IK-based reference generator (used in Attempt 4)
`diag_friction.py`	Friction diagnostic
`g1_pose.txt`	Spider pose joint angles
`summary.md`	Running development log
`main.tex / main.pdf`	This report